“Explainable Multi-Crop Plant Disease Diagnosis Using Convolutional Neural Networks on the TOM2024 Dataset”

PROJECT PHASE-1
GROUP - B13

Group Members

01 **VASISTA M**
CB.EN.U4AIE22134

02 **DHARMA THEJA**
CB.EN.U4AIE22137

03 **NANDANA GIREESH**
CB.EN.U4AIE22138

04 **SNEGA SRI A**
CB.EN.U4AIE22163

INTRODUCTION

- Plant diseases pose a significant threat to global agriculture, leading to major yield losses every year.
- Early and accurate detection of these diseases is essential for sustainable farming.
- Manual identification is **time-consuming, subjective, and requires domain expertise** .
- With the rise of Artificial Intelligence and Deep Learning, **image-based disease detection** has become faster and more reliable.
- This work explores **three deep learning models — CNN, ResNet, and EfficientNet —** to detect and classify diseases from the **TOM2024 dataset** .

PROBLEM STATEMENT

- Farmers often struggle to **identify diseases in their early stages** , leading to reduced crop productivity.
- Traditional diagnostic techniques are **manual, costly, and error-prone** .
- There is **no integrated intelligent system** capable of multi-crop and multi-disease classification.
- Hence, this project aims to **develop and compare robust deep learning models** to accurately detect plant diseases and integrate the solution into a **web-based application** for easy access.

OBJECTIVE

1. **Primary Goal** : Develop and compare three deep learning architectures (Custom CNN, ResNet-18, EfficientNet-B0) for automated classification of 30 crop pest and disease categories
2. **Performance Evaluation** : Assess model accuracy, precision, recall, and F1-scores to identify the most suitable architecture for agricultural deployment
3. **Interpretability Analysis** : Implement explainable AI techniques (Grad-CAM, LIME) to visualize which image regions influence model predictions
4. **Practical Application** : Create a reliable, deployable solution that can assist farmers in real-time pest and disease identification
5. **Benchmark Establishment** : Provide comparative insights on transfer learning vs. custom architectures for agricultural image classification tasks

LITERATURE REVIEW:

S. No.	Author(s) & Year	Method / Model Used	Dataset Used	Key Findings	Research Gap Identified	Novelty in Our Project
1	Mohanty et al. (2020)	CNN-based classifier	PlantVillage	Achieved 99% accuracy for 38 plant diseases in controlled images.	Limited to clean, single-crop dataset (lab images).	We used TOM2024 dataset containing field-level multi-crop images (tomato, maize, onion).
2	Too et al. (2020)	Transfer Learning (ResNet50, VGG16)	Custom crop dataset	Compared multiple CNN architectures for disease detection.	No clear model comparison framework across architectures.	We compared CNN, ResNet, and EfficientNet under identical conditions.
3	Ferentinos (2021)	Deep CNN	58,000 leaf images	High accuracy but lacked model explainability.	No interpretability tools like Grad-CAM or LIME used.	We implemented Grad-CAM & LIME to visualize disease-affected regions.
4	Lu et al. (2021)	ResNet-101	Rice and maize leaves	Deep residual models improved generalization.	Did not include efficient models like EfficientNet.	We integrated EfficientNet for lightweight, high-performance classification.
5	Agarwal et al. (2022)	MobileNetV2	Tomato disease dataset	Fast inference suitable for mobile applications.	Focused only on tomato, ignoring multi-crop generalization.	Our project supports 30 disease classes across 3 crops.

LITERATURE REVIEW:

S. No.	Author(s) & Year	Method / Model Used	Dataset Used	Key Findings	Research Gap Identified	Novelty in Our Project
6	Sharma et al. (2022)	DenseNet + CNN Ensemble	PlantVillage	Enhanced F1-score using hybrid ensemble.	No web or real-time deployment demonstrated.	We deployed a Flask-based web app for real-time disease prediction.
7	Kaur et al. (2023)	CNN with Data Augmentation	Custom leaf dataset	Improved training stability via augmentation.	Didn't address data imbalance properly.	We balanced dataset and evaluated macro/micro F1-scores to ensure fairness.
8	Li et al. (2023)	Vision Transformer (ViT)	Public crop dataset	Transformers improved classification on large datasets.	High computational cost; unsuitable for real-time apps.	Our approach achieves efficiency with EfficientNet-B0, ideal for real-time use.
9	Wang et al. (2024)	ResNet + SVM hybrid	Open-source leaf dataset	Improved disease discrimination for similar visual symptoms.	No integration with web or user interface.	We integrated model predictions directly into a web dashboard with treatment suggestions.
10	Patel et al. (2025)	YOLOv8 for Leaf Detection	Real-field tomato dataset	Detected diseased leaf regions using bounding boxes.	Focused on detection, not classification or prescription.	We provide end-to-end classification and disease remedy suggestions.

RESEARCH GAP

Limited Multi-Crop Datasets

Most previous studies focus on single-crop datasets (like PlantVillage), lacking field diversity and real-world variability.

→ *Our project uses the large, multi-crop TOM2024 dataset covering tomato, maize, and onion diseases.*

Inadequate Model Comparison

Few works have compared multiple deep learning architectures systematically on the other dataset.

→ *We perform a detailed comparative study using CNN, ResNet, EfficientNet.*

Lack of Explainability

Existing models act as "black boxes" with little focus on model interpretability.

→ *We integrate Grad-CAM and LIME visualization to make predictions transparent and explainable.*

Class Imbalance & Overfitting Issues

Previous research often overlooks imbalanced disease classes leading to biased models.

→ *We handle this through data augmentation, normalization, and performance metrics like F1-score and ROC-AUC.*

Absence of End-to-End Integration

Most studies stop at model evaluation without real-world deployment.

→ *We bridge this gap by developing a* ***web-based application*** *for plant disease diagnosis and visualization.*

01

METHODOLOGY

1.Dataset Overview

Dataset: TOM2024 Multi-Crop Dataset

Crops Covered:

- **Tomato** – 14 classes (12 diseases/healthy + 2 pests)
- **Onion** – 6 classes (5 diseases/healthy + 1 pest)
- **Maize** – 10 classes (7 diseases/healthy + 3 pests)
- **Labeled Images (processed):** 12,227
- **Total Classes:** 30 (including diseases, pests, and healthy leaves)
- **Image Format:** JPG/PNG, RGB color space
- **Resolution:** Resized to 224×224 pixels
- **Split Ratio:**
  a. 70% Training
  b. 20% Validation
  c. 10% Testing
- **Augmentation Techniques:** Random rotation, horizontal flip, and brightness adjustment to improve model generalization.

Category A.

Crop	Image categories	Number of classes	Name of pest / disease	Number of raw images	Number of processed images	Types of image processing
Tomato	Pest	2	Helicoverpa Harmigera	278	278	Cropping
			Tuta Absoluta	19	19	Cropping
	Diseases	12	Virosis	1542	1542	Cropping
			Healthy_Leaf	782	782	Cropping
			Healthy Fruit	552	552	Cropping
			Mite	390	390	Cropping
			Alternaria	322	322	Cropping
			Bacterial_floundering	252	252	Cropping
			Blossom_end_rot	118	118	Cropping
			Tomato_late_blight	74	74	Cropping
			Sunburn	51	51	Cropping
			Nitrogen_Excess	43	43	Cropping
			Fusarium	18	18	Cropping
			Alternaria_ Mite	8	8	Cropping
Onion	Pest	1	Caterpillars	879	879	Cropping
	Diseases	5	Fusarium	738	738	Cropping
			Healthy_Leaf	679	679	Cropping
			Alternaria	515	515	Cropping
			Virosis	203	203	Cropping
			Bulb_blight	30	30	Cropping
Maize	Pest	3	Spodoptera frugiperda (Fall Army Worm)_Activity	616	616	Cropping
			Spodoptera frugiperda	600	600	Cropping
			Aphids	4	4	Cropping
	Diseases	7	Stripe	2190	2190	Cropping
			Healthy_Leaf	585	585	Cropping
			Curvularia	256	256	Cropping
			Helminthosporiosis	159	159	Cropping
			Virosis	148	148	Cropping
			Rust	99	99	Cropping
			Abiotic_Diseases	77	77	Cropping

II. Data Preprocessing

Purpose:

To standardize and enhance image data before training the deep learning models (CNN, ResNet, EfficientNet).

Key Steps:

Image Resizing

- All images resized to **224 × 224 pixels** to ensure uniform input size for all models.
- Reduces computational load and maintains consistency.

Normalization

- Pixel values scaled between **0 and 1** for faster and stable training.

Data Augmentation

- Applied **rotation, flipping, zoom, and brightness** variations.
- Enhances dataset diversity and reduces overfitting.

Label Encoding

- Each disease category assigned a **unique numeric label** (0–29).
- Converts categorical data into a machine-readable form.

CNN :

Conceptual Understanding:

A CNN is inspired by the way the human brain processes visual information.

It learns directly from pixel data — understanding shapes, colors, and textures — to recognize disease symptoms like yellowing, spots, or mold patterns on leaves.

How it works conceptually:

1. **Feature Extraction:**
   - The model automatically detects simple features such as edges or color gradients in the first layers.
   - Deeper layers combine these to detect complex structures like leaf veins, texture patterns, and disease spots.
2. **Dimensionality Reduction:**
   - Pooling layers condense information by summarizing the most important visual cues, making the network efficient.
3. **Classification:**
   - Once the network has “seen” enough examples, it maps these visual cues to a specific disease class.
   - For example, brown patches may correspond to “Leaf Blight,” while yellow halos may point to “Bacterial Spot.”

Why CNN for Crop Diseases:

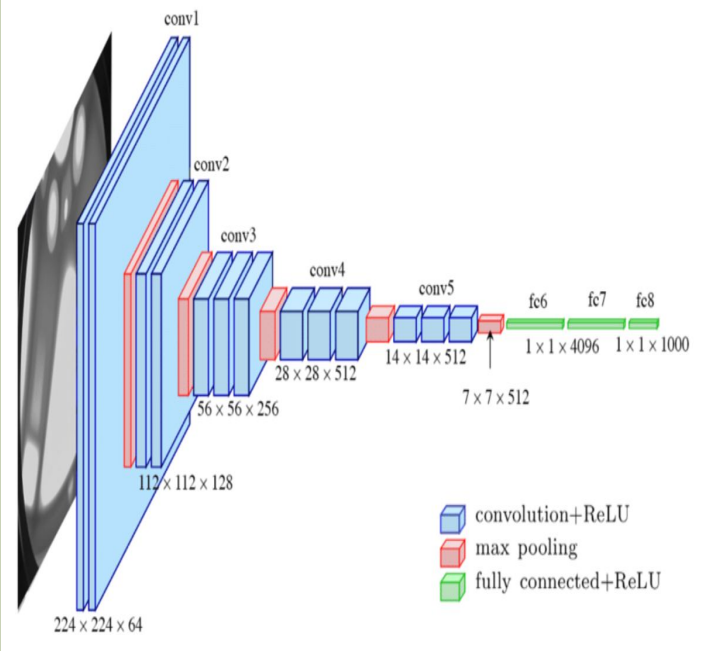
- Works directly on raw image pixels — no need for handcrafted features.
- Excels in recognizing spatial patterns typical of crop diseases.
- Forms the conceptual foundation for all modern vision models.

Architecture

Input RGB image → Conv Layers (32 → 64 → 128 → 256 → 256) → MaxPooling → Dropout → GAP → Fully Connected → Softmax Output

Key Components:

- **Input Layer:** Takes preprocessed RGB leaf images (224×224×3), normalized for stable training.
- **Convolutional Layers:** Extract low- to high-level features using 3×3 filters. Batch Normalization and ReLU activation stabilize and enhance learning.
- **Pooling Layers:** 2×2 MaxPooling after each block reduces spatial size, controlling overfitting and improving efficiency.
- **Dropout Layers:** Applied (0.3 and 0.5) to prevent overfitting on limited agricultural data.
- **1×1 Convolution:** Compresses feature channels while preserving depth-level information.
- **Global Average Pooling:** Aggregates spatial information, reducing parameters and improving generalization.



- **Fully Connected Layer:** Maps learned features to final class probabilities.
- **Output Layer:** Softmax activation produces the likelihood for each disease class.

Resnet-18:

Conceptual Understanding:

As CNNs grow deeper, they start to “forget” what was learned earlier due to vanishing gradients.

ResNet solves this by adding **shortcut (residual) connections** — allowing the model to “skip” certain layers and preserve important information.

Think of it like a memory system: instead of overwriting old knowledge, ResNet learns **what’s new** and adds it to what it already knows.

How it works conceptually:

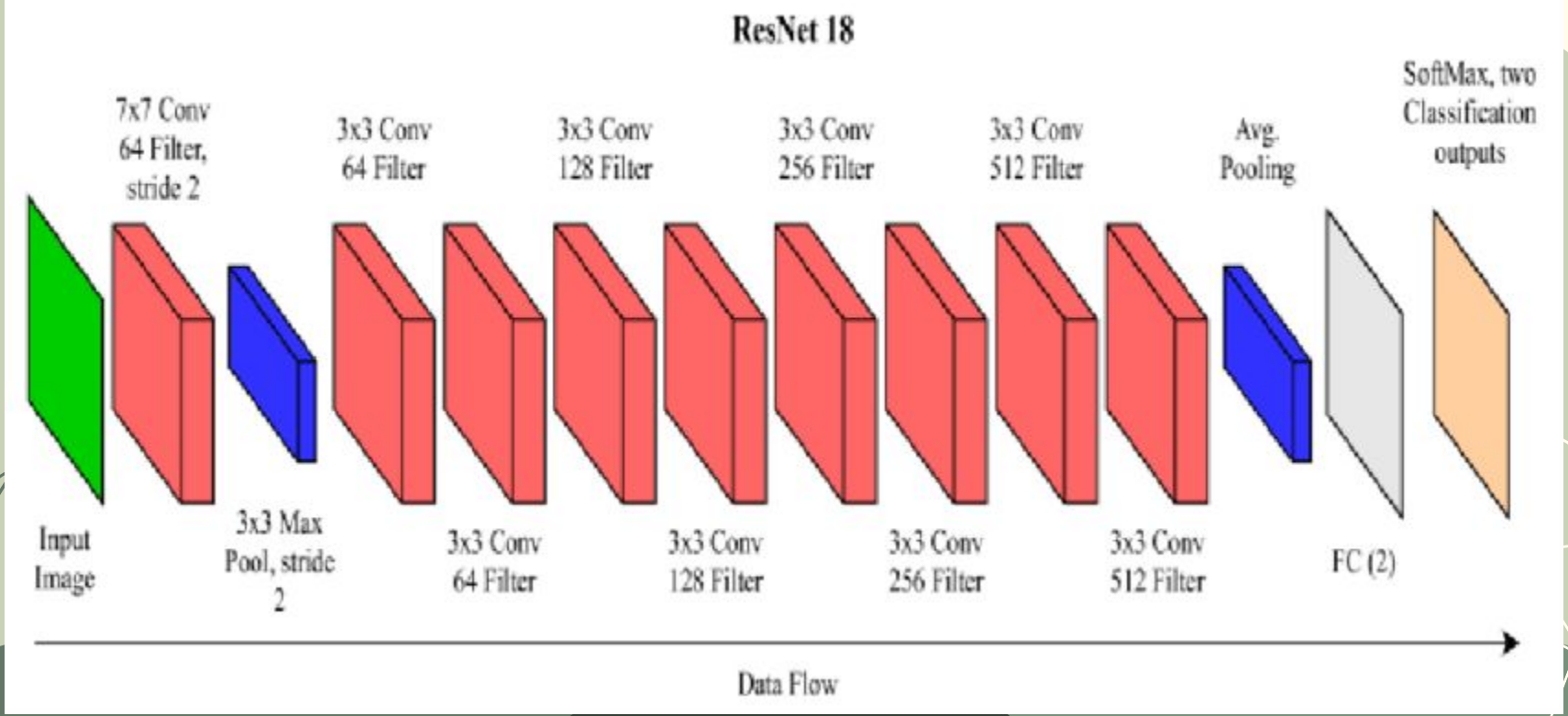
1. **Residual Blocks:**
   - Each block doesn’t learn the full transformation of data — it learns only the **difference (residual)** between input and output.
   - This helps the model focus on learning fine improvements rather than repeating the same features.
2. **Deep Representation Learning:**
   - By stacking many such blocks, ResNet can understand multiple levels of visual abstraction — from texture to leaf structure to overall plant health patterns.
3. **Interpretability:**
   - Because of its deep structure, ResNet identifies subtle disease cues (like fungal growth boundaries) that a standard CNN might miss.

Why ResNet for Crop Doctor:

- Enables much deeper and more accurate models.
- Learns both global and microscopic disease features.
- Ideal for detecting complex or overlapping disease symptoms.

Architecture:

Input → Conv(7×7) → BN → ReLU → MaxPool → Residual Blocks (Layer1–4) → AvgPool → FC(512→num_classes) → Softmax Output



Input → Conv(7×7) → BN → ReLU → MaxPool → Residual Blocks (Layer1–4) → AvgPool → FC(512→num_classes) → Softmax Output

Key Components:

- **Input Layer:** Accepts normalized RGB leaf images (224×224×3). Preprocessing ensures brightness and contrast consistency.
- **Initial Convolution Block:** A large **7×7 convolution** with 64 filters followed by **BatchNorm**, **ReLU**, and **3×3 MaxPooling** reduces spatial resolution and captures broad leaf structures.
- **Residual Blocks:**
  - Consist of paired **3×3 convolutions** with skip connections that directly add the input to the output (`F(x) + x`).
  - Four main stages (Layer1 → Layer4) progressively increase depth:
    - Layer1: 64 filters
    - Layer2: 128 filters
    - Layer3: 256 filters
    - Layer4: 512 filters
  - These blocks enable very deep learning without gradient loss, preserving both low- and high-level leaf features.
- **Batch Normalization & ReLU:** Applied throughout to stabilize gradients and improve training efficiency.
- **Global Average Pooling:** Converts feature maps into a single 512-dimensional vector summarizing the entire image's learned features.
- **Fully Connected Layer:** Replaced with `Linear(512 → num_classes)` to output 30 disease probabilities.
- **Output Layer:** Softmax activation produces class probabilities, predicting the disease with the highest likelihood.

EfficientNet-B0:

Conceptual Understanding:
EfficientNet was designed to answer a key question:
“How can we make deep learning models both **accurate** and **efficient** at the same time?”

Unlike CNN or ResNet, which grow deeper or wider manually, EfficientNet **scales** its structure in a balanced way — expanding depth, width, and input resolution together using a mathematical scaling rule.

How it works conceptually:

1. **Compound Scaling Principle:**
   - Instead of arbitrarily adding layers or filters, EfficientNet uses a formula that proportionally increases network dimensions to maximize learning without wasted computation.
2. **Smart Building Blocks:**
   - Uses **MBConv (Mobile Inverted Bottleneck Convolution)** blocks to capture fine-grained features efficiently.
   - Integrates **Squeeze-and-Excitation** modules that let the model “pay more attention” to important parts of the leaf image (like the infected area).
3. **Balance of Speed and Accuracy:**
   - EfficientNet achieves near state-of-the-art accuracy with a fraction of the parameters of ResNet.
   - It is compact enough to run in mobile apps — essential for the Crop Doctor app’s real-time diagnosis goal.

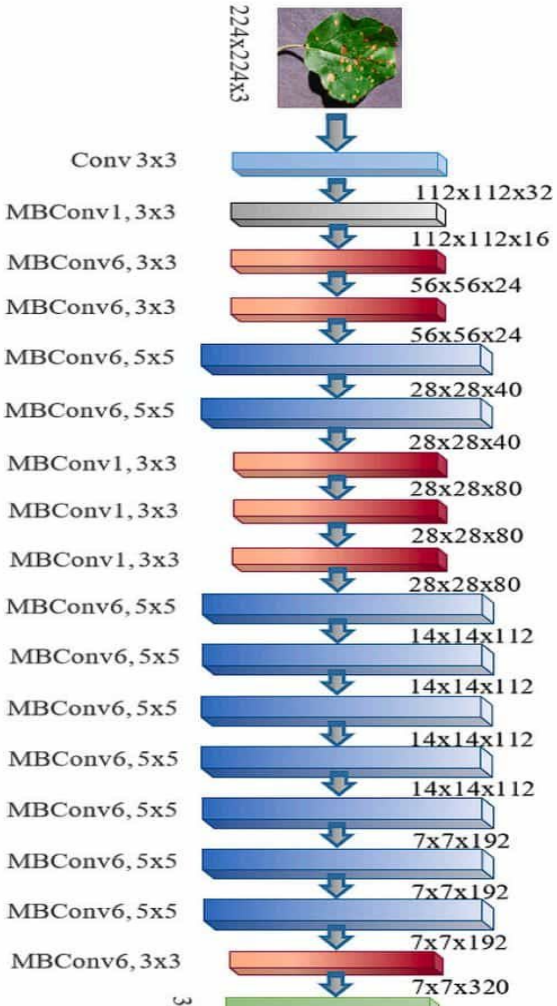
Why EfficientNet for Crop Doctor:

- Perfect balance between computation and accuracy.
- Can be deployed easily on phones or low-power devices.

Architecture:

Key Components:

- **Input Layer:** Accepts normalized RGB images (224×224×3) consistent with EfficientNet preprocessing.
- **Base Network:** Pretrained EfficientNet-B0 backbone (ImageNet weights) with frozen parameters to retain low-level visual features.
- **MBConv Blocks:** Capture edge patterns and disease textures using inverted residuals and squeeze-and-excitation layers.
- **Global Average Pooling:** Converts 2D feature maps into global descriptors, reducing parameters and preventing overfitting.
- **Custom Classifier Head:**
  - Dropout(0.5)
  - Linear(1280→512) + SiLU + BatchNorm + Dropout(0.3)
  - Linear(512→num_classes) for 30-class prediction.
- **Output Layer:** Softmax activation generates class probabilities for



RESULTS

Web Application Integration

Purpose:

To deploy the trained EfficientNet model into a **user-friendly web interface** for real-time plant disease detection.

Implementation Overview:

- **Framework:** Flask (Python-based lightweight web framework).
- **Frontend:** HTML, CSS for design and user interaction.
- **Backend:** Flask routes handle image upload, prediction, and results rendering.
- **Model Integration:**
  - Trained **EfficientNet model (.pth)** loaded in backend.
  - Prediction and suggestion generation performed using PyTorch.

- **Output Display:**
  - Upload plant leaf image → Model predicts disease → Webpage displays:
    - Predicted disease name
    - Recommended treatment or control measure

🌱 Plant Disease Detection

Browse... 776961890_1678878199433_640x480-bsogmf.jpg

Upload & Predict

🌿 Prediction Result

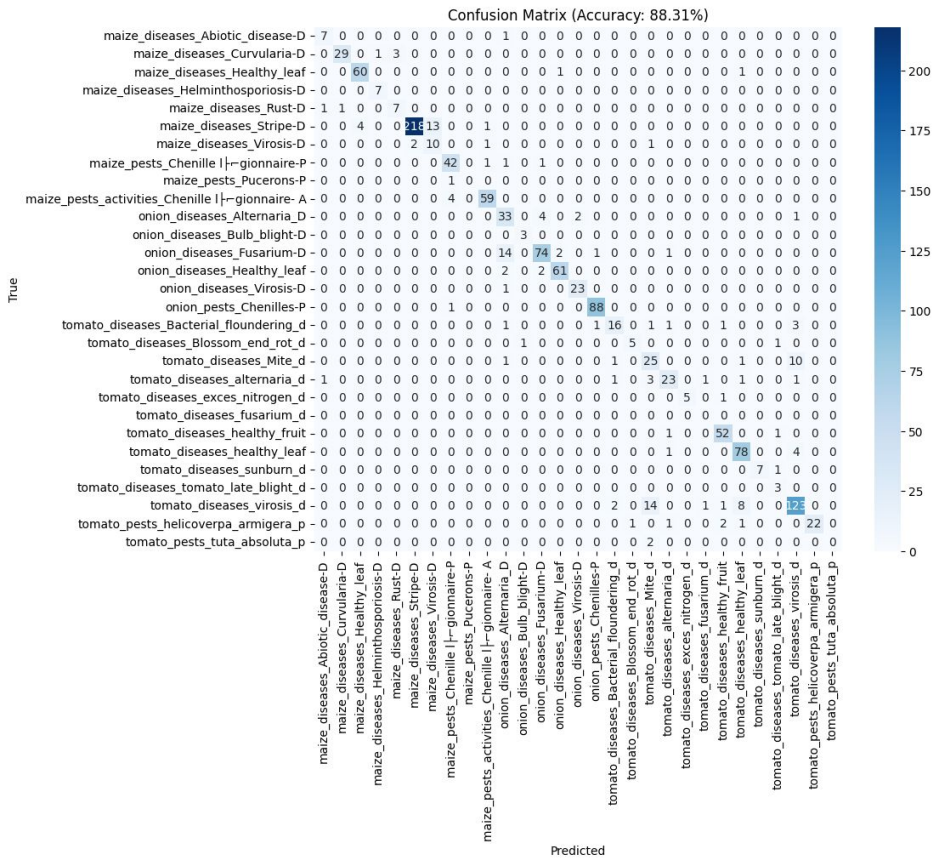
Disease: tomato_diseases_virosis_d

Uploaded Image

💡 **Suggestion: Remove infected plants. Control insect vectors (whiteflies, aphids). Avoid planting susceptible varieties in the same field.**

🔄 Try Another Image

EfficientNet MODEL-Confusion Matrix



High Overall Accuracy:
Achieved a **test accuracy of 88.31%** with a **low test loss of 0.4863**, showing strong generalization across all 30 crop disease classes.

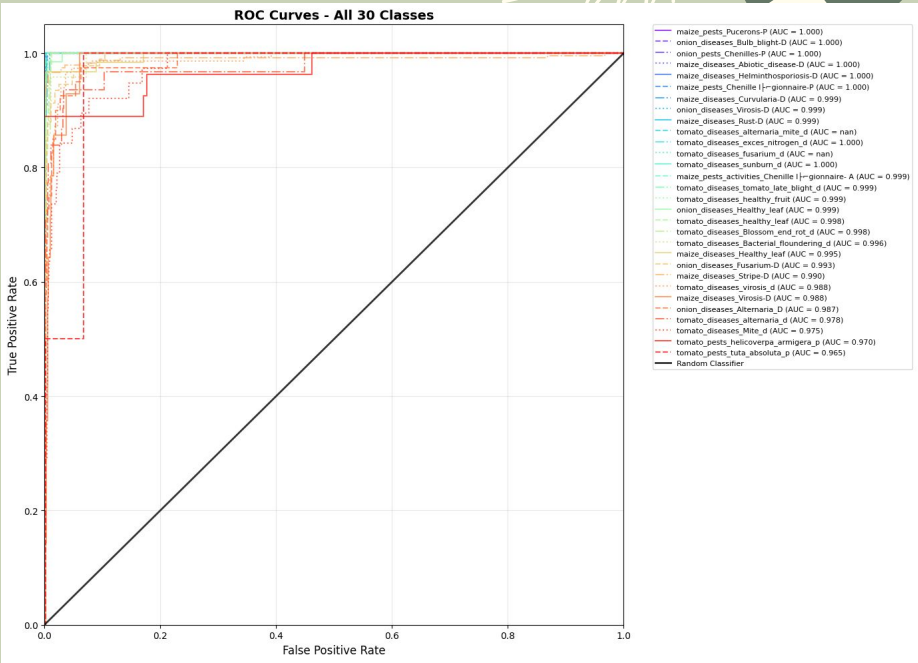
Balanced Class Performance:
Macro-average **F1-score = 0.7571** and weighted **F1-score = 0.8860**, indicating consistent performance even in multi-class conditions.

Strong Detection for Major Classes:
High precision and recall for key classes such as

- *maize_diseases_Stripe-D (F1 = 0.95)*
- *onion_pests_Chenilles-P (F1 = 0.98)*
- *tomato_diseases_healthy_leaf (F1 = 0.90)*

Challenges with Rare Classes:
Minor classes like *maize_pests_Pucerons-P* and *tomato_pests_tuta_absoluta_p* had lower scores due to **limited training samples** (class imbalance).

Loss Curve and ROC Curve



Grad-Cam And Lime Outputs

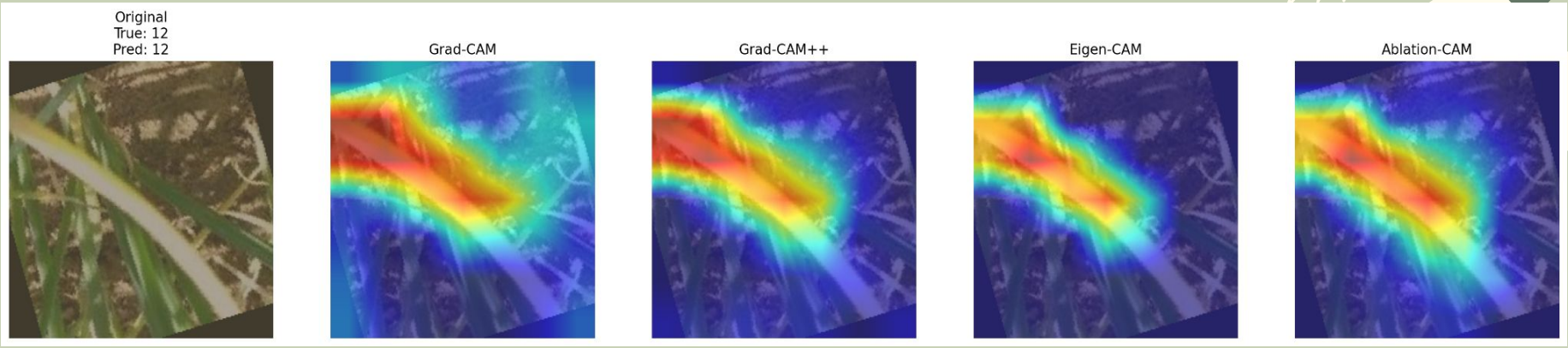


TABLE I
PERFORMANCE OF CNN

Train Acc	Val Acc	Test Acc	Prec.	Rec.	F1
82.4%	77.8%	78.9%	0.79	0.77	0.78

TABLE II
PERFORMANCE OF RESNET-18

Train Acc	Val Acc	Test Acc	Prec.	Rec.	F1
92.45%	85.00%	82.75%	0.84	0.83	0.83

TABLE III
PERFORMANCE OF EFFICIENTNET-B0

Train Acc	Val Acc	Test Acc	Prec.	Rec.	F1
97.71%	90.15%	88.31%	0.89	0.88	0.88

Comparison of Models

Model	Validation Accuracy	Test Accuracy	ROC-AUC	F1-Score	Interpretability	Deployment Readiness
CNN	77.8%	78.9%	0.88	0.78	Low	Medium
ResNet-18	85.0%	82.75%	0.92	0.83	Medium	High
EfficientNet-B0	**90.15%**	**88.31%**	**0.96**	**0.88**	**High**	**Excellent**

CONCLUSION

- **EfficientNet Achieved Superior Performance:**
  Among the three models, **EfficientNet** recorded the **highest accuracy (88.31%)** with robust F1-scores, proving its efficiency in feature extraction and generalization.
- **Deep Learning Enhanced Early Disease Detection:**
  The models effectively identified diseases across **maize, onion, and tomato** crops, supporting early diagnosis and reducing crop loss.
- **Explainability Improved Model Trust:**
  Techniques like **Grad-CAM and LIME** helped visualize affected leaf regions, ensuring model transparency and interpretability.
- **Real-Time Deployment Success:**
  Integration with a **Flask-based web application** allowed real-time image uploads, disease prediction, and remedy suggestions — bridging research and field use.